Q1------slip4----------

```
import heapq
GOAL = (1, 2, 3, 0)
def h(s):
    return sum(abs(i//2 - GOAL.index(x)//2) +
               abs(i%2 - GOAL.index(x)%2)
               for i, x in enumerate(s) if x != 0)
def moves(s):
    z = s.index(0)
    r, c = divmod(z, 2)
    ans = []
    for dr, dc in [(-1,0),(1,0),(0,-1),(0,1)]:
        nr, nc = r+dr, c+dc
        if 0 <= nr < 2 and 0 <= nc < 2:
            j = nr*2+nc
            t = list(s)
            t[z], t[j] = t[j], t[z]
            ans.append(tuple(t))
    return ans
def astar(start):
    pq = [(h(start), 0, start, [])]
    seen = set()
    while pq:
        f, g, s, path = heapq.heappop(pq)
        if s in seen:
            continue
        seen.add(s)
        if s == GOAL:
            return path + [s]
        for t in moves(s):
            if t not in seen:
                heapq.heappush(pq, (g+1+h(t), g+1, t,
                                    path+[s]))
    return None
start = (1, 0, 3, 2)
sol = astar(start)
if sol:
    for s in sol:
        print(s[:2], s[2:])
    print("Moves:", len(sol)-1)
else:
    print("No solution")
```

```
Q2-----slip4----------------------------------------
tree = [[3, 5], [6, 9]]
def minimax(node, depth, is_max):
   if depth == 2:
      return node
   if is_max:
      return max(minimax(x, depth+1, False)
            for x in node)
   return min(minimax(x, depth+1, True)
         for x in node)
def alphabeta(node, depth, alpha, beta, is_max):
   if depth == 2:
      return node
   if is_max:
      value = float('-inf')
      for x in node:
         value = max(value, alphabeta(
            x, depth+1, alpha, beta, False))
         alpha = max(alpha, value)
         if alpha >= beta:
            break
      return value
   else:
      value = float('inf')
      for x in node:
         value = min(value, alphabeta(
            x, depth+1, alpha, beta, True))
         beta = min(beta, value)
         if alpha >= beta:
            break
      return value
print("Minimax:", minimax(tree, 0, True))
print("Alpha-Beta:", alphabeta(
   tree, 0, float('-inf'), float('inf'), True))
```

Slip3---q1----

```
import java.util.*;

class DFS {
   static Map<Character, List<Character>> graph = new HashMap<>();
   static Set<Character> visited = new HashSet<>();

   static void dfs(char node) {
      visited.add(node);
      System.out.print(node + " ");
      for (char next : graph.get(node)) {
         if (!visited.contains(next)) {
            dfs(next);
         }
      }
   }

   public static void main(String[] args) {
      graph.put('A', Arrays.asList('B', 'C', 'E'));
      graph.put('B', Arrays.asList('A', 'D'));
      graph.put('C', Arrays.asList('A', 'F'));
      graph.put('D', Arrays.asList('B', 'E'));
      graph.put('E', Arrays.asList('D', 'F', 'A'));
 graph.put('F', Arrays.asList('C', 'E'));

      System.out.print("DFS Traversal: ");
      dfs('A');
   }
}
```

Slip 3------q2-------

```
from sklearn.svm import SVC
# Training data
X = [
 [1, 2],
 [2, 3],
 [5, 5]
]
# Class labels
y = [0, 0, 1]
# Test input
X_test = [[4, 4]]
# Create Non-Linear SVM using RBF kernel
model = SVC(kernel='rbf')
# Train the model
model.fit(X, y)
# Predict the class of test input
prediction = model.predict(X_test)
print("Training Data:")
print(X)
print("\nClass Labels:")
print(y)
print("\nTest Input:")
print(X_test)
print("\nPredicted Class:")
print(prediction)
```